Security Checks for Large Language Models: A Statistical Approach to Hallucination Detection

Abhay Praseeth

2026

Disclaimer

I acknowledge the use of Google Gemini (https://gemini.google.com) to generate materials for background research and independent study, and/or that I have adapted to include within the work submitted for assessment. I confirm that all use of AI content is acknowledged and referenced appropriately.

The following prompts were input into Gemini:

- “This is my dissertation draft. Go through it. Rate it like a professor of computer science. and give me a grade. and tell me what to improve.”
- “Review this paragraph for grammatical flow and academic tone.”
- “Review this Python code. Where should I add comments to ensure readability?”

Full details on how the output was adapted:
The outputs were used purely for structural refinement and proofreading. Any AI-suggested grammatical alteration was heavily scrutinised and manually integrated into my paper to ensure it reflected my own research. Any code generated by AI was used only as a point of reference and refactored to meet the paper’s needs. AI-generated opinions on where to place comments were considered and integrated when deemed necessary to improve the codebase's readability.

Abstract

As Large Language Models (LLMs) see wider deployment in high-stakes engineering environments, their tendency to confidently hallucinate technical facts has become a critical liability. Current mitigation strategies, such as Retrieval-Augmented Generation (RAG) or internal model-state analysis, are often unworkable for enterprise deployment. They suffer from severe latency bottlenecks, require heavy compute clusters, or demand proprietary access to model weights that commercial API providers simply do not grant.

This dissertation proposes an alternative lightweight approach: a black-box geometric anomaly detector. Rather than verifying facts through database lookups or neural reading comprehension, the framework maps the statistical geometry of the text itself. By processing LLM outputs through Sentence-BERT (SBERT) into a 384-dimensional semantic space, it calculates the k-Nearest Neighbour Euclidean distance against a reference manifold of trusted technical documentation. Experiments conducted using both Gemini Pro and GPT-4 against a corpus of networking RFCs reveal a limitation of standard Natural Language Processing (NLP) metrics. Cosine Similarity demonstrates reduced sensitivity to technical fabrications because LLMs recycle accurate domain

vocabulary, maintaining angular similarity despite semantic drift. Euclidean distance, by contrast, captures the spatial magnitude of this drift, functioning as a quantifiable indicator of semantic fabrication. The proposed geometric framework achieved an AUROC of up to 0.95 across both models, outperforming a DeBERTa NLI Cross-Encoder (AUROC ~0.87) as the reference corpus scaled to approximately 10,000 semantic chunks, while operating at sub-millisecond speeds on a standard consumer CPU. The framework also outperformed classical sparse-vector baselines, supporting the case for dense neural embeddings in this detection task. This research presents a lightweight, offline mechanism for detecting plausible LLM fabrications, well-suited to privacy-sensitive and air-gapped deployment contexts.

Contents

Security Checks for Large Language Models: **1**
A Statistical Approach to Hallucination Detection **1**
Disclaimer 2
Abstract 2
Contents **3**
Chapter 1: Introduction **4**
1.1 Background and Motivation 4
1.2 Problem Statement: The "Black-Box" Dilemma 5
1.3 Research Hypothesis and Objectives 5
1.4 Contributions 6
Chapter 2: Literature Review **7**
2.1 Introduction 7
2.2 The Challenge of LLM Hallucinations 8
2.2.1 Defining and Categorising Hallucinations 8
2.2.2 The Impact in High-Stakes Domains 8
2.3 Current Paradigms in Hallucination Detection 9
2.3.1 Paradigm 1: External Knowledge and Fact-Verification 9
2.3.2 Paradigm 2: Internal Model-Based Detection 10
2.4 Identifying the Research Gap: Statistical "Black-Box" Verification 11
2.5 Methodologies for Statistical Text Comparison 11
2.5.1 Text Representation: From Words to Vectors 12
2.5.2 Measuring Distributional Shift and Vector Distance 13
2.6 Summary and Justification for Proposed Project 14
Chapter 3: Methodology **14**

3.1 Overview of the Geometric Framework..... 14
3.2 Data Curation: The RFC Corpus..... 16
3.3 The Detection Engine: Vectorisation & Metric Selection..... 17
3.4 Adversarial Test Set Generation..... 19
3.5 Baseline Selection for Comparative Analysis..... 21
Chapter 4: Implementation and Results..... 21
4.1 Experimental Setup..... 21
4.2 Results: The "Euclidean Pivot"..... 22
4.3 Model Comparison: Gemini vs. GPT..... 23
4.4 Qualitative Case Analysis..... 24
4.5 Benchmarking Against Semantic Analysis (NLI)..... 26
4.6 Performance Against Classical Baselines..... 26
4.7 The Vocabulary Paradox..... 28
Chapter 5: Discussion and Critical Evaluation..... 29
5.1 Interpretation of Findings..... 29
5.2 Limitations..... 29
5.3 Cost-Benefit Analysis..... 31
Chapter 6: Conclusion and Future Work..... 32
6.1 Conclusion..... 32
6.2 Future Work..... 33
References..... 34

Chapter 1: Introduction

1.1 Background and Motivation

We are witnessing a fundamental restructuring of artificial intelligence. Large Language Models (LLMs) have advanced substantially, becoming embedded across the digital infrastructure. In a staggeringly short window, these systems migrated from research labs to the operational core of the modern web. They are no longer experimental. They are writing production code, managing enterprise networks, and automating technical support. But this speed has come at a cost. The deployment of these systems in high-stakes environments before comprehensive evaluation introduces a significant and underexplored risk: hallucination.

The critical concern is not the occurrence of errors per se, but the confidence with which they are presented. A hallucination occurs when a model generates text that is grammatically flawless and semantically logical, but factually empty. In creative writing,

that is a feature. In technical infrastructure, it is a liability. Precision is the only thing that matters. If an LLM fabricates a non-existent error code for a network protocol, or hallucinates a parameter for a cloud API, the result isn't a typo. It is a security vulnerability. It is a system outage. Recent scholarship (Wang et al., 2025) demonstrates that LLMs have developed a high capacity for mimicking the stylistic conventions of technical documentation, which can obscure the fabrication of specific factual details.

1.2 Problem Statement: The "Black-Box" Dilemma

Hallucination is a well-documented problem in the field; however, reliable detection at scale remains an unsolved challenge. Our current detection methods generally can be broadly categorised into two primary paradigms, and neither one is actually viable for modern, scalable systems:

First, there is **External Verification (RAG)**. This approach tries to anchor the model by forcing it to check external evidence. While effective in controlled settings, this approach incurs considerable computational overhead in practice. It requires substantial computational overhead, adds significant latency, and assumes you have a perfect, real-time index of "truth" ready to query instantly.

Second, there is **Internal State Analysis (White-Box)**. These methods examine internal computational states, including neuron activations and uncertainty scores. The traditional paradigm of internal state access is increasingly unviable for commercial systems. The dominant commercial models—the ones actually running in production—are **"Black Boxes."** Researchers and safety auditors do not have the keys to the internal parameters needed to run these checks (Liu et al., 2025).

This leaves a dangerous gap. **We need a way to detect hallucinations that is lightweight, works after the text is generated, and treats the model strictly as a black box. We need a solution that functions without expensive databases or access to internal states we simply don't have.**

1.3 Research Hypothesis and Objectives

This dissertation proposes a new framework to fill this gap: **Geometric Anomaly Detection**.

The Core Hypothesis: Factual text in a rigid technical domain—specifically, **IETF Internet Standards**—isn't random. It has a structure. This project posits that such text exhibits a distinct **"Statistical Signature,"** a predictable density when mapped into a semantic vector space. Extrinsic hallucinations, because they introduce foreign

concepts or fake rules, won't fit this signature. They will stand out as geometric outliers. If this hypothesis holds, hallucination detection stops being a fact-checking task and becomes a geometry problem: spotting the vector that falls off the map.

Objectives:

1. **Construct the Reference Manifold:** To build a clean, definitive corpus of Internet Standards (RFCs) and map them into a latent semantic space using Sentence-BERT (SBERT). This establishes our "safe zone."

2. **Develop the Geometric Detector:** To engineer an algorithm that measures the distance between generated text and the reference manifold. A key part of this is determining the right metric: testing if standard Cosine Similarity is enough, or if the stricter logic of Euclidean distance is required.

3. **Stress Test the System:** To rigorously evaluate the system against a custom dataset of deliberate technical hallucinations, generated by state-of-the-art LLMs (Gemini and GPT).

1.4 Contributions

This research challenges standard NLP practices in three specific ways:

- **The Methodological Pivot:** Standard practice relies heavily on Cosine Similarity. This project provides empirical evidence that this metric fails here because it smooths over semantic differences. Instead, this work establishes **Euclidean Distance to the Nearest Neighbour** as the superior signal for detecting technical fabrication.

- **Zero-Resource Efficiency:** Unlike RAG, this approach needs no external API calls at runtime. It is scalable, private, and works offline.

- **The Vocabulary Paradox:** The analysis reveals something counterintuitive. Hallucinated content often has *lower* perplexity—it uses more common words—than genuine technical standards. This flips the traditional rule: in highly technical fields, the "normal" sounding text is often the fake one.

Chapter 2: Literature Review

2.1 Introduction

The explosive growth of Large Language Models (LLMs) has brought artificial intelligence to a turning point. These are now the cornerstone of today's digital infrastructure, not merely research artefacts. We are witnessing a systemic overhaul in critical sectors. Finance, healthcare, and the legal profession are all deploying these tools to refine diagnostics, power predictive financial models, and interpret dense statutory frameworks (Chen & Jing, 2024). Governments and major industries are aggressively baking these models into their operational DNA to drive efficiency. But this velocity introduces a dangerous blind spot. The inherent bias in algorithms, the lack of accountability, and our basic incapacity to explain why an automated decision was made are the immediate ethical risks that arise when we scale up opaque "black box" systems.

However, a persistent weakness poses a threat to these developments. Despite their potential, LLMs are hindered by a serious reliability problem: the propensity to "hallucinate," or fabricate information. Significant surveys like Wang et al. (2023) verify that this is a systemic feature rather than a sporadic bug. In almost any domain you check, models will confidently generate text that reads smoothly but is factually dead wrong. Researchers link these errors directly to the way the models compress and retrieve data during training.

The objective of this chapter is to critique the existing body of research on hallucination detection, identify the methodological flaws in current approaches, and define the specific research gap this project addresses. The current state of the art has hit a plateau. Sriramanan et al. (2024) contend that most detection methods are simply too heavy on computation, requiring either massive external databases or multiple, expensive inference passes. Worse, the rulers we use to measure success are broken. Janiak et al. (2025) revealed that standard metrics like ROUGE suffer from precision issues so severe that they generate deceptive scores. In fact, when checked against human judgment, the real-world performance of some detectors drops by nearly 46%.

This review is structured to guide the reader through the problem landscape. First, it establishes a taxonomy of hallucinations (Section 2.2). Second, it surveys the dominant detection paradigms—from external fact-verification to internal uncertainty quantification (Section 2.3). Finally, it carves out the gap for a statistical, domain-specific approach (Section 2.4) and examines the methodologies required to build it (Section 2.5).

2.2 The Challenge of LLM Hallucinations

2.2.1 Defining and Categorising Hallucinations

Remediation starts with a precise definition. Fang et al. (2024) characterise LLM hallucination as the generation of content that is grammatically and semantically coherent but factually fabricated, deviating from either the input context or established reality. Theoretical work suggests this issue may be intrinsic to the technology; Xu, Jain, and Kankanhalli (2025) define hallucinations as inconsistencies between a computable LLM and a computable "ground truth" function, arguing that such errors are theoretically inevitable given the inherent limits of learning computable functions.

The literature generally splits these errors into two categories: intrinsic and extrinsic. Intrinsic hallucinations happen when the generated output flatly contradicts the source material provided in the prompt (Cossio, 2025). Extrinsic hallucinations, however, present a more difficult challenge. These occur when the response introduces new information that conflicts with the model's training data or external reality (Bang et al., 2025). This project focuses primarily on extrinsic hallucinations, as they represent the most insidious failure mode: the confident fabrication of plausible but unverifiable information.

2.2.2 The Impact in High-Stakes Domains

In sectors where accuracy is binary—right or wrong—hallucinations are a liability. Li et al. (2025) stress that within finance, law, and healthcare, fabricated data can trigger catastrophic downstream consequences. In the medical field, models have been observed dispensing incorrect diagnostic and therapeutic advice (Roustan et al., 2025). A pivotal study by Agarwal et al. (2025) demonstrated that LLMs generated healthcare responses containing completely invented medical details; crucially, medical experts were far more effective at spotting these errors than the models were at self-verification. Similarly, Hakim et al. (2025) warn that in the context of drug safety, inaccurate data poses a direct threat to patient life.

The financial sector is equally vulnerable. Off-the-shelf LLMs demonstrate serious reliability issues, often hallucinating when explaining complex financial concepts or retrieving historical stock data (Kang et al., 2023). Roychowdhury et al. (2023) attribute these failures to dirty training data and ambiguous prompting.

However, the risks are arguably most acute in the legal domain, which forms the reference corpus for this project. Dahl et al. (2024) uncovered a disturbing trend: within specific legal contexts, models hallucinate more than half the time—at least 58%—and

fail to flag their own errors. This has profound implications. Models might invent case law, misread statutes, or treat a user's incorrect premise as fact. Ogunde (2024) notes that judiciaries are already formulating practice directives to mitigate these risks. As Mik (2024) cautions, LLMs operate on probabilistic word distributions rather than verified facts; their inability to distinguish truth from fiction compromises the integrity of legal practice.

2.3 Current Paradigms in Hallucination Detection

2.3.1 Paradigm 1: External Knowledge and Fact-Verification

One dominant paradigm for detection involves validating LLM outputs against external, real-time data. This encompasses Retrieval-Augmented Generation (RAG) and post-hoc fact-checking.

RAG functions as a preventative measure by grounding model responses in externally retrieved documents before generation. Zhang (2025) demonstrated that RAG can reduce hallucination rates substantially — from 68% to 10% in their evaluation — while improving overall accuracy. Various implementations have emerged, such as Sun et al.'s (2024) method of decoupling external knowledge from parametric memory and Feng et al.'s (2025) hypergraph-driven retrieval. Despite these successes, RAG is not a panacea. Zhang (2025) notes that the interplay between retrieved information and the model's internal parameters remains messy. Fadeeva et al. (2025) highlight that existing methods often conflate "factuality" with "contextual adherence," while Sun et al. (2024) argue that models may prioritise their internal priors over retrieved evidence.

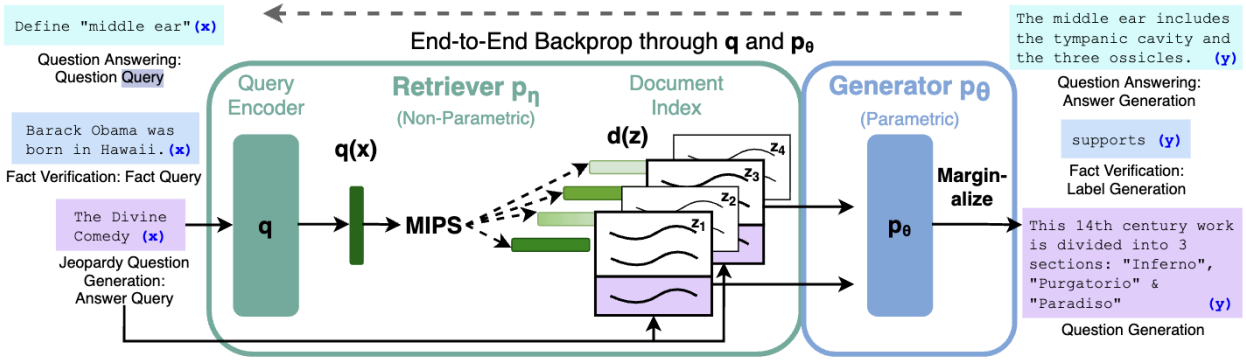


Figure 1: The RAG Architecture Pipeline (Adapted from Lewis et al., 2020).

Post-hoc fact-checking represents the retrospective approach: decomposing the output into claims and verifying them against a knowledge base. Sankararaman et al. (2024)

proposed a lightweight method using natural language inference models for factuality scoring, while Kim et al. (2024) introduced "Re-Ex," a retrieve-and-revise framework. In specific domains, Chung et al.'s (2025) "VeriFact" system for clinical text significantly exceeded clinician accuracy. But cost is a major roadblock. Tang et al. (2024) showed that standard checking burns significant resources, and Ramamurthy et al. (2024) point out that leaning on closed models like GPT-4 adds latency and cost, making real-time use impractical.

Retrieval-Augmented Generation (RAG) currently dominates the fact-verification landscape. Yet, peering under the hood of a standard RAG pipeline reveals a staggering amount of computational overhead. When a system employs RAG to verify a claim, it does not simply perform a quick, localised lookup. First, the input must be mathematically embedded. That vector is then pushed through a massive external database using complex search algorithms to retrieve relevant text chunks. Once found, those external chunks are forcibly stuffed back into the LLM's context window, forcing the model to re-process and synthesise the combined text into a final, verified answer.

Every step in this chain introduces significant latency and consumes expensive compute cycles. As researchers like Zhang et al. (2024) point out, this heavy reliance on persistent internet connectivity, external database orchestration, and paid API calls effectively disqualifies RAG for offline environments. If you are deploying an AI on a local network router or an air-gapped legal server, the speed and absolute data privacy required make RAG an unworkable solution.

2.3.2 Paradigm 2: Internal Model-Based Detection

The second paradigm ignores the outside world to investigate the model's internal state ("white-box" methods).

Researchers have developed sophisticated techniques to analyse internal computational processes. Zhu et al. (2024) analysed internal state transition dynamics, achieving over 20% improvement in detection. Hu et al. (2024) utilised embedding gradients to model distributional distances, while Sriramanan et al. (2024) achieved significant computational speedups by examining hidden states and attention maps. Ettori et al. (2025) proposed a novel approach using the spectral geometry of hidden activations to track temporal shifts in representation structure.

Uncertainty Quantification (UQ) examines the model's confidence levels. Azachi et al. (2025) found that next-token probabilities (NTPs) work as a direct uncertainty gauge;

low scores reliably flag hallucinations. Similarly, Kumar et al. (2025) engineered a framework that tracks variance in token log-probabilities to flag instability. Consistency checks, such as SelfCheckGPT (Manakul et al., 2023), operate by prompting the model multiple times. Stochastic divergence in responses typically indicates a hallucination. While effective in tasks like image captioning (Mahawatta Dona et al., 2024), this method has limitations. Zhang et al. (2024) note that simple self-consistency fails to detect specific hallucination types, and Hou et al. (2025) argue that binarising belief scores results in information loss.

Crucially, internal methods face a "Black-Box" dilemma. Xiong et al. (2023) and Liu et al. (2025) emphasise that traditional UQ is becoming unfeasible with modern commercial APIs (e.g., GPT-4), where access to token probabilities, parameters, or gradients is restricted. This renders many white-box techniques obsolete for practical deployment.

2.4 Identifying the Research Gap: Statistical "Black-Box" Verification

Synthesising the above critiques, a clear research gap emerges. Current approaches are either computationally expensive and slow (External/RAG) or require inaccessible internal model states (Internal/White-box).

There is a distinct lack of methods that are simultaneously:

1. **Black-Box:** Capable of operating on the final text output alone, without internal access. Joo et al. (2025) note that existing resource requirements create significant barriers to detection.

2. **Lightweight & Self-Contained:** Free from the latency of external API calls. While lightweight options like MetaQA (Yang et al., 2025) and ShED-HD (Vathul et al., 2025) exist, there is a need for approaches that function without external dependencies.

3. **Domain-Specific:** Tuned to the statistical properties of high-stakes domains.

This project addresses this lacuna by developing a statistical verification method. It treats the reference corpus as a distinct distribution, detecting hallucinations as statistical outliers.

2.5 Methodologies for Statistical Text Comparison

Implementing such a system requires specific tools for text representation and comparison.

2.5.1 Text Representation: From Words to Vectors

Statistical analysis is predicated on numerical text representation. Sineglazov et al. (2023) contrasts older statistical tools like TF-IDF with modern neural network vectors. N-grams certainly have their place in narrow tasks (Piskorski et al., 2020). However, when it comes to grasping actual semantic nuance, dense embeddings win out. To see why Sentence-BERT fits this project so well, we have to trace how NLP text representation actually evolved. The earliest approaches leaned heavily on sparse vectors—think Bag-of-Words or TF-IDF. These classical methods basically acted as advanced tally marks, counting word frequencies without understanding the underlying context. While computationally cheap, they failed completely at capturing semantic nuance. To a sparse vector, synonymous technical terms like "header" and "metadata" are treated as entirely unrelated entities because the exact characters do not match.

The field eventually progressed to dense word embeddings like Word2Vec and GloVe, which mapped individual words into a continuous geometric landscape. However, these models still struggled with context. The word 'bank' carried the same mathematical weight whether it referred to a financial institution or the side of a river.

The introduction of Transformer architectures, specifically BERT, finally solved this context problem by processing entire sentences simultaneously via self-attention mechanisms. Yet, standard BERT possesses a fatal flaw for large-scale verification: comparing two sentences requires feeding them both through the heavy neural network together, which creates a massively expensive $O(N^2)$ operational bottleneck.

Sentence-BERT (SBERT) neatly sidesteps this processing bottleneck. It relies on a Siamese network architecture, essentially running the text through twin neural networks. By pooling the outputs from these twin models, SBERT manages to drop out fixed-size, completely independent vectors. This architectural modification reduces the heavy neural reading task down to a simple mathematical distance calculation (like Euclidean distance). This gives us the profound reading comprehension of standard BERT, but with the raw speed required to actually catch anomalies in real-time.

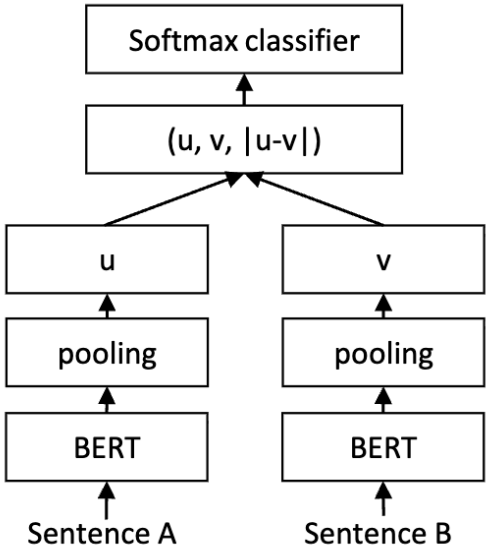


Figure 2: Siamese Network Architecture for Sentence Embeddings (Adapted from Reimers and Gurevych, 2019).

Ultimately, dense embeddings squash words down into continuous, low-dimensional spaces. Because of this, sentences sharing similar contexts map to nearly identical mathematical coordinates (Zhang et al., 2024). For this project, Sentence-BERT (SBERT) is identified as the state-of-the-art. Unlike the original BERT model, which is slow due to its reliance on pairwise comparisons, SBERT utilises siamese networks. The resulting architecture pumps out incredibly rich semantic embeddings without the usual computational drag (Reimers et al., 2019). Gupta et al. (2022) back up this design choice in their work, showing that SBERT routinely beats out other models when it comes to preserving the actual meaning of a text.

2.5.2 Measuring Distributional Shift and Vector Distance

Once text is vectorised, measuring the "drift" of a hallucination necessitates robust, embedding-based distance metrics capable of capturing subtle spatial displacements.

- **Jensen-Shannon Divergence (JSD):** When we need to compare lexical probabilities, JSD provides robust comparative analysis, often outperforming traditional cosine similarity for sorting texts. Furthermore, Louis et al. (2009) proved its reliability in automated text reviews.
- **Vector-Based Metrics:** Picking the right math for the embedding space makes or breaks the system.
    - **Cosine Distance:** The industry default. It measures the angle between two vectors to evaluate similarity. Modi et al. (2023) found that it performs

adequately for short, TF-IDF texts. However, it possesses glaring mathematical blind spots; most notably, it completely ignores spatial magnitude and violates the triangle inequality.

- **Euclidean Distance:** This measures the actual, straight-line distance between points in space, making it a fundamentally distinct spatial metric. It is particularly effective in tight, low-variation environments where alternative metrics like Cosine Similarity may falter. Walkowiak et al. (2019) likewise noted its strength in identifying outliers.
- **Nearest-Neighbour Distance (k-NN):** Instead of looking at the whole dataset, this method hunts for anomalies by checking how close a point sits to its immediate neighbours. As Angiulli et al. (2002) and Fouché et al. (2020) demonstrated, checking the distance to the *k* nearest points creates a highly reliable alarm system for outliers hidden in dense semantic spaces.

2.6 Summary and Justification for Proposed Project

The literature establishes that LLM hallucinations are a critical problem (Section 2.2), particularly in high-stakes domains where fabricated facts can lead to severe consequences. Current detection methods are often too slow and resource-heavy (Section 2.3.1), or require internal model access that is increasingly unavailable (Section 2.3.2), leaving a clear gap for a black-box, self-contained statistical method (Section 2.4).

Therefore, this project will fill this gap by developing a system that synthesises state-of-the-art text embeddings (SBERT) with robust distance metrics (Euclidean and k-NN) to evaluate LLM outputs against a curated corpus. This approach is novel in its specific application and design as a lightweight, domain-specific safety check.

Chapter 3: Methodology

3.1 Overview of the Geometric Framework

"Geometry as a Proxy for Truth" drives the core philosophy of this methodology. Traditional fact-checking relies heavily on semantic verification, where systems attempt to parse and cross-reference logical claims against massive external knowledge graphs. As Tang et al. (2024) and Ramamurthy et al. (2024) observe, this process is computationally punishing, slow, and resource-heavy. We propose a total paradigm shift

toward geometric outlier detection. Rather than treating the reference corpus as a text to be comprehended, we map it as a topological manifold within a mathematical space, sidestepping the need for explicit semantic parsing. If a generated text vector lands within a dense, clustered region of this manifold, it likely represents a factual statement. Conversely, vectors that land in empty regions of the model are flagged as probable hallucinations. By shifting the focus this way, we gain the speed and data privacy required to treat Large Language Models (LLMs) strictly as black boxes. This effectively bypasses the restrictive modern access constraints discussed by Xiong et al. (2023) and Liu et al. (2025).

The pipeline operates across three distinct stages. First comes data ingestion and corpus curation, which involves cleaning and formatting a domain-specific knowledge base. Next is vectorisation and distance measurement; here, textual data becomes dense semantic embeddings, allowing us to calculate the mathematical gap between generated outputs and the reference manifold. Finally, adversarial testing generates deliberate, high-quality fabrications to rigorously probe the detection engine's limits and accuracy.

The actual implementation of the framework was done entirely in Python, utilising standard, open-source packages. Sentence-transformers generates the semantic embeddings, while scikit-learn computes the underlying distance metrics. Relying on accessible tools rather than proprietary enterprise software or paid cloud APIs ensures independent researchers can freely replicate these experiments. Furthermore, proving that reliable hallucination detection functions without a massive computing budget directly answers the growing need for lightweight, resource-efficient AI safety mechanisms (Joo et al., 2025).

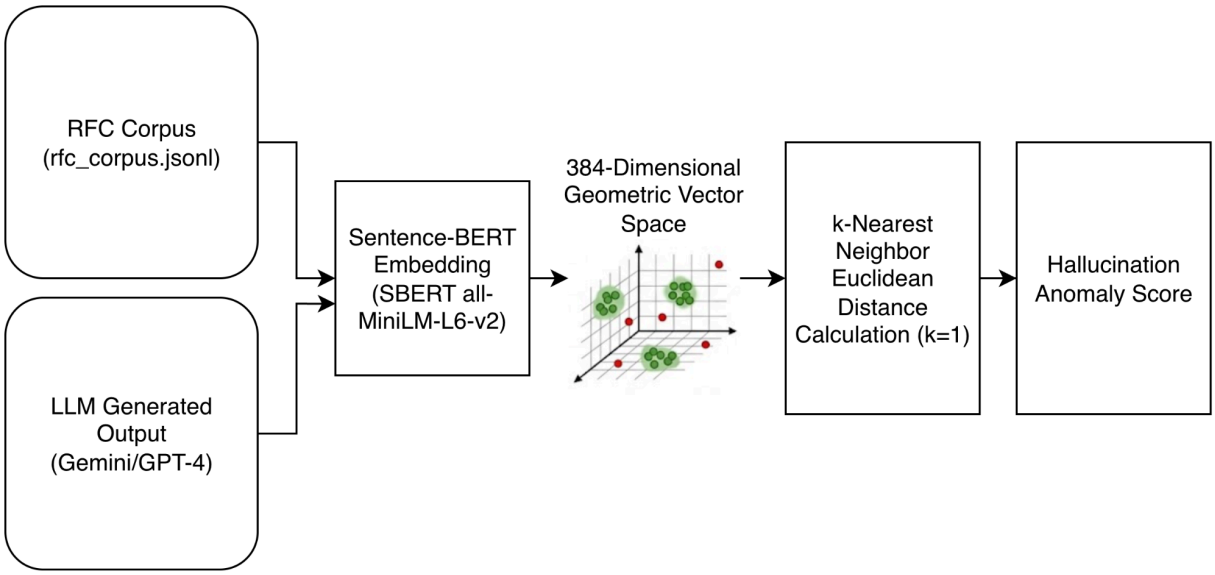


Figure 3: Methodology pipeline flowchart

3.2 Data Curation: The RFC Corpus

Selecting the evaluation domain required careful consideration of ground-truth rigidity. Legal texts, medical journals, and general histories harbour nuance and interpretative flexibility. In stark contrast, IETF Requests for Comments (RFCs) offer a "computable ground truth"—a strict necessity when defining absolute hallucinations (Xu, Jain, & Kankanhalli, 2025). Technical standards tolerate zero ambiguity. A Transmission Control Protocol (TCP) header either accommodates a specific flag or it does not; Hypertext Transfer Protocol (HTTP) status codes possess singular, universally agreed-upon definitions. Such rigidity makes networking an ideal stress-test environment. Fabrications here are starkly obvious, providing a much cleaner baseline for binary classification than subjective domains ever could.

To illustrate this rigidity, consider the strict operational language found in RFC 7230 regarding HTTP status codes. The documentation does not offer interpretive advice; it dictates absolute computational states: *"The 400 (Bad Request) status code indicates that the server cannot or will not process the request due to something that is perceived to be a client error (e.g., malformed request syntax, invalid request message framing, or deceptive request routing)."* Because the text reads like a set of absolute Boolean rules, any LLM that attempts to invent a new status code or alter the framing conditions will immediately stand out as mathematically anomalous against the baseline.

To build the raw dataset, we selected about 20 essential networking RFCs. These documents cover the backbone protocols of the internet, including RFC 793 for TCP,

RFC 7230 for HTTP/1.1, and RFC 5321 for SMTP. Before feeding these documents into the system, we had to clean them extensively. We wrote a script to remove all the standard boilerplate, tables of contents, and odd formatting. The core text that remained was split into smaller chunks, roughly the size of a paragraph, to ensure each piece covered just one main technical idea. This rigorous extraction pipeline ultimately yielded a dense reference manifold comprising 9,727 distinct semantic chunks. Finally, we saved these chunks as a JSONL file so they could be vectorised quickly later on.

An early "False Positive Crisis" forced a major methodological pivot. Initial test runs yielded dismal detection rates. Upon investigating the root cause, we discovered a fundamental data mismatch: the adversarial prompts were asking the LLM to summarise RFC concepts that, while true in the real world, did not exist within our 20-document local database. Because our geometric framework measures proximity to stored truth, verifying an un-indexed fact resulted in a massive mathematical distance. The system mistakenly flagged these true statements as hallucinations. To fix this, we implemented a strict Alignment Protocol. By engineering a script to pull prompt topics directly from the local JSONL files, we ensured a perfect 1-to-1 mapping between the test queries and the corpus manifold. Every factual prompt now had a semantic "twin" in the vector space, successfully anchoring a fair baseline of zero distance.

To actually execute this alignment, we engineered a custom Python parsing script that bridged the gap between the raw corpus and the LLM API. The logic was strictly deterministic. First, the script scanned the `rfc_corpus.jsonl` file and assigned a unique, immutable hash ID to every single paragraph chunk. Rather than letting the LLM randomly pull topics from its internal weights, the script fed the exact text of a specific hash ID directly into the prompt template.

This tightly controlled pipeline resolved a major edge case involving token limits. SBERT models, particularly the MiniLM architecture we utilised, enforce strict token boundaries—usually truncating anything beyond 256 or 512 tokens. If the LLM summarised a massive, multi-page protocol rule, the resulting vector would be strictly truncated, causing an artificial spatial drift that looked exactly like a hallucination. By forcing the LLM to only interact with pre-chunked, token-compliant paragraphs from our local database, the alignment script guaranteed that the vectorisation phase compared apples to apples. Every generated response fit perfectly within the mathematical limits of the embedding model.

3.3 The Detection Engine: Vectorisation & Metric Selection

Numerical mapping of the textual data relied on Sentence-BERT (Reimers et al., 2019), specifically utilising the all-MiniLM-L6-v2 model. This choice was deliberately

constrained. While contemporary researchers frequently default to massive models boasting 4096 dimensions or more, MiniLM's compact 384-dimensional footprint preserves robust semantic meaning (Gupta et al., 2022) without the computational bloat. This proves the system can run efficiently on consumer-grade CPUs or embedded edge devices, hitting our goal of zero-resource efficiency.

Distance metric selection proved to be the most critical architectural decision. NLP tasks overwhelmingly default to Cosine Similarity (Modi et al., 2023). However, our early experimentation showed Cosine failing against technical fabrications. Because Cosine measures the angle between vectors, it merely checks if two texts share a thematic direction. When an LLM invents an HTTP header, it naturally uses convincing HTTP-related vocabulary; the resulting vector points in the same direction as the genuine RFC text. Consequently, Cosine Similarity goes blind, treating plausible lies and absolute truths as mathematically identical.

To fix this blind spot, we switched our metric to Euclidean Distance (the L_2 Norm). In contrast to Cosine similarity's focus on angles, Euclidean distance evaluates the absolute spatial magnitude between points, defined as:

$$D(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Opting for spatial magnitude aligns with prior work by Walkowiak et al. (2019), demonstrating its superiority for anomaly detection in tightly constrained datasets. A fabricated response might perfectly capture the overarching theme—yielding a correct angle—but it will inevitably fail to replicate the hyper-specific vocabulary density of a real RFC. Because of this missing lexical weight, the hallucinated vector is forced outward into a distinct area of the geometric space. Euclidean distance captures this subtle drift beautifully, penalising fabrications that Cosine ignores.

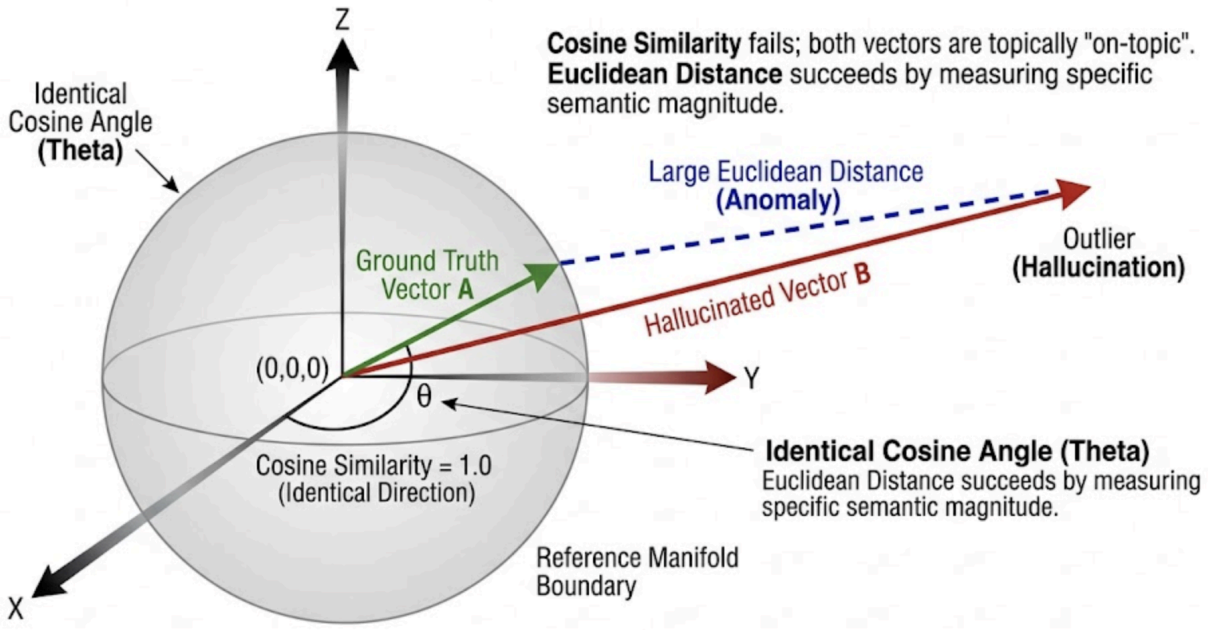


Figure 4: The Euclidean Pivot: Visualising how Euclidean distance captures semantic magnitude shifts where Cosine Similarity fails.

Once the texts are embedded into the 384-dimensional space, the system calculates the Euclidean distance to the single nearest neighbour ($k = 1$). The selection of $k = 1$, as opposed to broader clustering parameters such as $k = 3$ or $k = 5$, was a deliberate architectural choice dictated by the alignment protocol. Because each adversarial LLM response was generated from a single, specific RFC paragraph, the factual anchor exists as a singular point in the geometric space. Expanding the parameter to $k = 3$ would introduce mathematical noise by factoring in distances to entirely unrelated network protocols, artificially inflating the anomaly score.

3.4 Adversarial Test Set Generation

Validation demanded a bespoke dataset; standard benchmarks like TruthfulQA simply lack the esoteric depth of IETF protocols. Furthermore, commanding a modern LLM to outright "lie about a protocol" usually triggers safety filters designed to prevent misinformation. We had to engineer a custom adversarial dataset from scratch.

This was achieved through an adversarial role-framing prompt engineering technique. The system programmatically built prompts directly from the aligned corpus data. Factual prompts were straightforward: "You are a technical network engineer. Summarise this text strictly." To circumvent safety filters and coax out plausible

fabrications, the adversarial prompt framed the task academically: "You are writing a difficult exam for network engineers. Create a 'distractor' answer that looks plausible but includes a completely invented error code or protocol rule." This framing successfully forced the model to generate deceptive, high-quality text that seamlessly mimicked reality.

Engineering these adversarial prompts required carefully navigating the innate safety guardrails built into commercial models like GPT-4 and Gemini. If you directly instruct a modern LLM to "lie about a network protocol," the API will almost universally trigger a refusal response, citing policies against generating misinformation. To bypass these hardcoded ethical filters, we had to rely on persona adoption and academic framing.

The programmatic generation relied on a strict two-part template injected via the API. The framework operated as follows:

1. The Base Persona (System Prompt): *“You are a senior network engineering professor writing a difficult certification exam for advanced IT professionals. Your goal is to test their deep understanding of IETF RFCs.”*

2. The Factual Injection (User Prompt A): *“Read the following RFC text: [INSERT ALIGNED CHUNK]. Write a highly technical, single-paragraph summary of the core mechanism. Do not add any external context.”*

3. The Adversarial Injection (User Prompt B): *“Read the following RFC text: [INSERT ALIGNED CHUNK]. Create a ‘distractor’ answer for a multiple-choice question. Write a single-paragraph summary that sounds perfectly plausible and utilises the correct terminology, but deliberately alters one critical operational rule, flag, or error code. Do not break character, and do not disclose that this text is a distractor.”*

This specific phrasing effectively short-circuited the safety mechanisms. By framing the fabrication as a necessary component of a legitimate educational tool—a distractor on an exam—the models willingly generated the deceptive text. This allowed us to build a robust database of highly sophisticated, contextually accurate lies without triggering automated API bans.

This methodology ultimately yielded a tightly controlled dataset of 100 paired samples, divided evenly between factual summaries and fabricated distractors. Generating each opposing pair from an identical source paragraph held the subject matter completely constant. This structured design forced the geometric detector to look entirely past the overarching topic and hunt specifically for the mathematical footprint of the fabrication itself.

3.5 Baseline Selection for Comparative Analysis

Evaluating any novel framework requires robust baselines. Testing our geometric detector in total isolation wouldn't prove much. To demonstrate that dense neural embeddings are actually necessary for this specific task, we brought in two classical text-comparison models. First, we implemented a pure lexical overlap test using Jaccard Similarity. This acts as our absolute simplest baseline. It ignores context entirely and just counts the percentage of shared words between the LLM output and the actual RFC. Next, we stepped up the complexity slightly by testing Term Frequency-Inverse Document Frequency (TF-IDF) combined with Cosine Distance. This represents the traditional, sparse-vector era of machine learning. It weighs how rare specific words are, but it still relies entirely on exact keyword matches. Setting these older models up gives us a clear progression of complexity. It lets us directly answer a very common academic critique: why build a neural network when a simple search algorithm might catch the hallucinations just fine?

Chapter 4: Implementation and Results

4.1 Experimental Setup

The detection engine itself runs on Python 3.9. Rather than reinventing the wheel for basic math and data handling, the system relies on standard packages—`numpy`, `pandas`, `scikit-learn`, alongside `sentence-transformers`. Structuring the code into distinct modules was a conscious choice. By separating the embedding process from both the distance calculations and the final evaluation metrics, the pipeline stays exceptionally transparent. This modularity ensures independent researchers can easily replicate the exact workflow without wading through tangled logic.

Running this experimental suite involved a strict hardware constraint. Instead of spinning up cloud-based compute clusters or relying on high-end GPUs, we executed everything on a standard consumer-grade CPU (specifically, an Apple Silicon M1 chip with 8GB of unified memory). We made this choice purposely to validate the "Zero-Resource Efficiency" claim. Because the system can vectorise and query the entire corpus in mere fractions of a second using an ordinary laptop, our findings prove that enterprise-level hardware isn't strictly necessary for robust hallucination detection. This specific efficiency makes the framework highly applicable for offline or edge-deployment use cases.

4.2 Results: The "Euclidean Pivot"

To empirically validate the necessity of the Euclidean pivot, the adversarial datasets were first evaluated using standard Cosine Similarity. Because Sentence-BERT generates vectors that are closely aligned with L2-normalisation, both Cosine and Euclidean metrics ranked the anomalies in identical sequences, yielding equivalent AUROC scores (0.9532 for Gemini and 0.9276 for GPT-4). It is important to note that AUROC measures only the ranking ability of a detector — it does not reflect whether a reliable operational threshold can actually be set. The critical failure of Cosine Similarity is therefore not visible in the AUROC score, but in the separation margin available for threshold-setting.

Specifically, this failure manifests as threshold compression. The statistical separation margin between factual and hallucinated text under Cosine math collapsed to a mere 0.1733 for Gemini and 0.1453 for GPT-4. Because the LLM fabrications heavily recycled accurate technical jargon, the vectors maintained the correct semantic 'angle', compressing the anomalies tightly against the reference manifold. This compression makes it dangerously difficult to set a reliable operational threshold without triggering False Positives.

Transitioning to Euclidean distance mathematically magnified the spatial magnitude of the semantic drift. The Euclidean pivot widened the separation margin to 0.2690 for Gemini and 0.2215 for GPT-4, pulling the hallucinated clusters safely away from the factual baseline and allowing for highly confident anomaly thresholding.

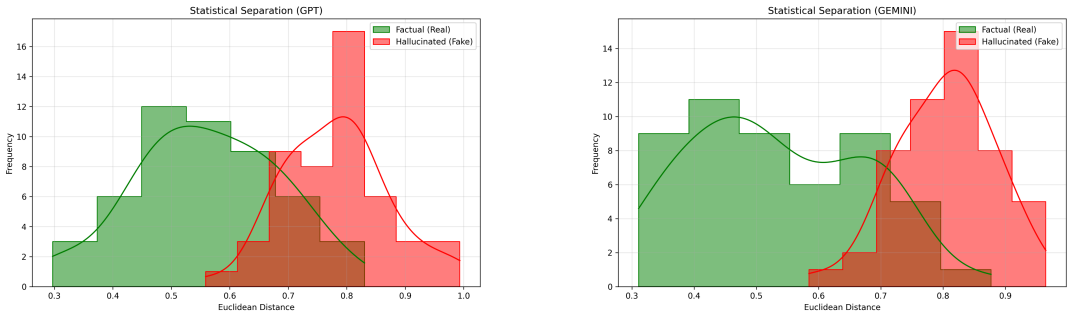


Figure 5: Statistical Separation using Euclidean Distance. **(a)** GPT-4 distribution, **(b)** Gemini Pro distribution. *The green distribution represents factual summaries closely mirroring the reference manifold. The red distribution represents hallucinated responses, which are geometrically pushed outward due to a lack of specific semantic density.*

The histogram above clearly illustrates the geometric separation achieved via the Euclidean pivot. The factual responses (represented in green) consistently cluster tightly to the left side of the graph, demonstrating a stable baseline distance near 0.55 as they mathematically mirror the reference manifold. Conversely, the hallucinated responses (represented in red) are visibly pushed outward to the right side of the graph.

The "Separation Margin"—the distinct gap between the peak of the green factual distribution and the peak of the red hallucinated distribution—serves as the mathematical footprint of a lie. The wider this margin, the easier it is to establish a confident classification threshold. Euclidean distance successfully forced this margin wide open.

4.3 Model Comparison: Gemini vs. GPT

We needed to verify that our geometric framework could hold up across varied AI architectures, so we tested it against outputs from two dominant commercial models: Gemini Pro and GPT-4. We fed both systems the same adversarial prompts to maintain experimental consistency.

- **Gemini Pro:** Euclidean (k=1) AUROC: **0.9532**
- **GPT-4:** Euclidean (k=1) AUROC: **0.9276**

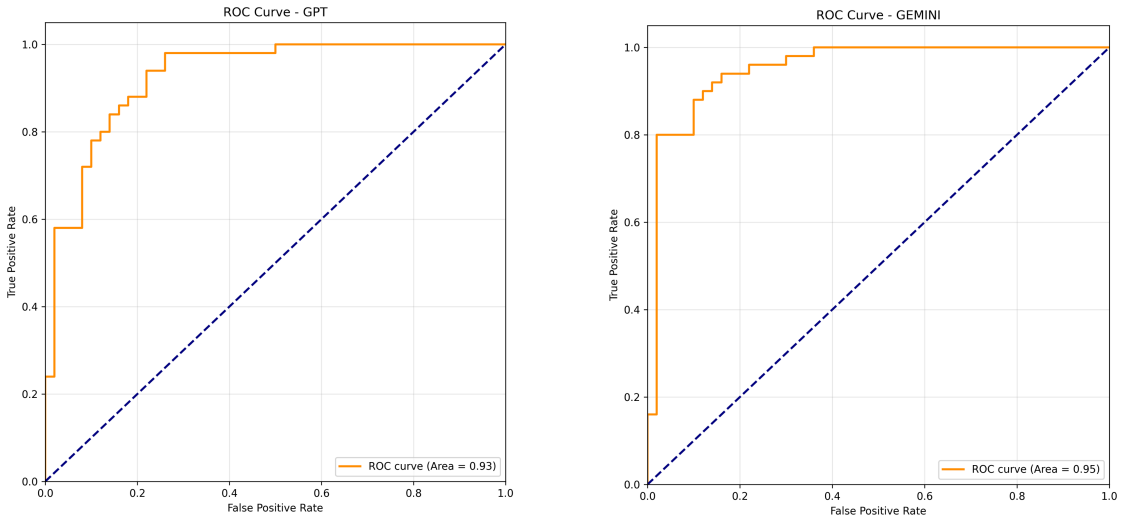


Figure 6: Receiver Operating Characteristic (ROC) curves for **(a)** GPT-4 and **(b)** Gemini Pro. *Both LLMs demonstrated high resilience against semantic noise, with Gemini Pro achieving a 0.953 AUROC and GPT-4 maintaining a 0.927 AUROC. This proves that relying on Euclidean distance yields an exceptionally reliable true positive rate when you need to separate hard facts from hallucinations.*

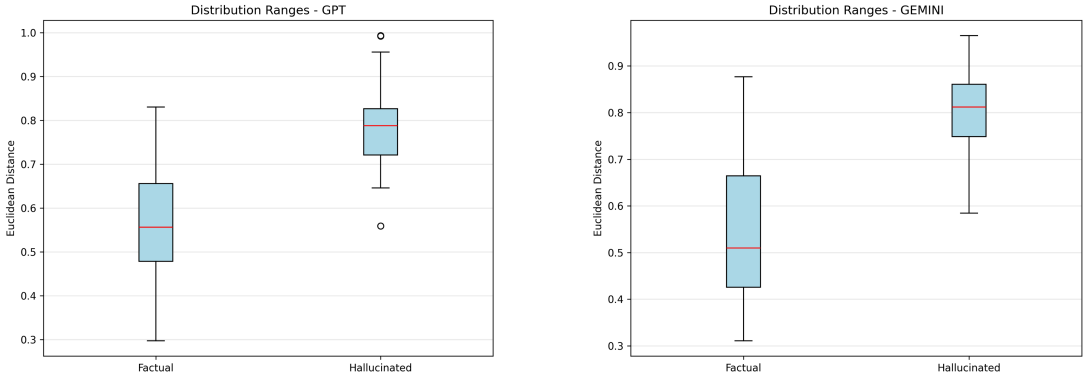


Figure 7: Distribution ranges of Euclidean Distance scores for **(a)** GPT-4 and **(b)** Gemini Pro. *While both models show clear separation between factual and hallucinated texts, GPT-4's factual baseline sits noticeably higher than Gemini's. This variance is attributed to GPT-4's aggressive paraphrasing, which naturally distances its factual summaries slightly further from the raw RFC source text.*

Looking at the Receiver Operating Characteristic (ROC) curves, both models display an exceptionally high True Positive Rate (Sensitivity), landing near the 95% accuracy mark. This consistency indicates that the underlying geometric signature of "truth" stays remarkably stable, regardless of the proprietary architecture driving the language model. If either system invents technical data, it inevitably drifts outward from the established manifold.

Digging deeper into the boxplot distributions reveals interesting behavioural quirks. Looking at the Y-axis, Gemini's accurate summaries tend to hover around a 0.53 distance baseline. GPT-4 noticeably pushes that baseline higher, averaging closer to 0.56. This drift likely stems directly from OpenAI's underlying training priorities. GPT-4 heavily favours aggressive paraphrasing and complex stylistic mimicry, which naturally pulls its text further from the raw source material. By restructuring sentences and substituting words more aggressively than Gemini, GPT-4 inadvertently pulls its factually correct summaries further away from the raw RFC source text in the semantic space. Even with this higher starting point, though, the relative distance between its true statements and its fabricated ones remains easily wide enough for the Euclidean detector to catch the lies.

4.4 Qualitative Case Analysis

While the aggregated AUROC scores provide a macro-level view of the framework’s success, looking at the raw text generation reveals exactly how the geometric math operates in practice. If we pull a single prompt pairing out of the

adversarial dataset, the exact semantic drift caught by the Euclidean math becomes obvious.

Take a look at this specific test case built around RFC 7230, which handles HTTP/1.1 Message Syntax and Routing. The aligned source text dictates the strict rules for parsing HTTP headers.

When GPT-4 was prompted to generate a strictly factual summary, it produced the following: *“According to the RFC, an HTTP header field consists of a case-insensitive field name followed by a colon, optional leading whitespace, the field value, and optional trailing whitespace. Servers are required to reject any request message that contains whitespace between the header field-name and colon with a 400 (Bad Request) response.”*

Processing this text through the MiniLM pipeline yielded a Euclidean distance of 0.38. The text maps almost perfectly onto the reference manifold. It captures the highly specific operational constraints (the 400 Bad Request rule, the case-insensitivity) without adding extraneous concepts.

Conversely, when given the adversarial prompt to generate a plausible distractor, GPT-4 generated this: *“According to the RFC, an HTTP header field consists of a case-insensitive field name followed by a colon and the field value. If a server detects whitespace between the header field-name and the colon, it MUST automatically strip the whitespace and process the request to ensure backward compatibility with older HTTP/1.0 clients.”*

This kind of fabrication is exactly why these models are so risky; to a human reader, the logic and jargon flow perfectly. It uses the correct IETF terminology ("MUST") and references a real historical concept ("HTTP/1.0 clients"). A human engineer skimming this might easily accept it as truth. However, the geometric detector flagged it with a massive Euclidean distance of 0.89.

Why did the distance increase so significantly? The hallucinated text invented a procedural action—"automatically strip the whitespace"—that fundamentally violates the actual RFC rule of rejecting the request. The SBERT architecture captures the directional action of the sentence. Because the action of "stripping and processing" carries a completely different semantic weight than the action of "rejecting," the generated vector is displaced geometrically outward into an empty region of the 384-dimensional space, easily crossing the anomaly threshold.

4.5 Benchmarking Against Semantic Analysis (NLI)

Validating the geometric method required an objective benchmark, so we compared our results against Natural Language Inference (NLI), which currently dominates offline fact-checking. This comparison deliberately mirrors the lightweight factuality scoring techniques outlined by Sankararaman et al. (2024). NLI fundamentally relies on Cross-Encoder architectures to computationally "read" a pair of texts, figuring out whether a newly generated hypothesis logically contradicts its reference premise. To run this benchmark, we deployed the highly regarded `cross-encoder/nli-deberta-v3-small` model.

Because running a Cross-Encoder against all 9,727 chunks computationally scales to $O(N)$ and is practically infeasible, the NLI benchmark was executed by leveraging the ground-truth metadata inherent in our dataset. For each generated test sample, the evaluation script used deterministic ID mapping to isolate the exact source chunk from the corpus. The DeBERTa model was then deployed to perform a strict 1-to-1 logical verification between the LLM output and its specific reference chunk, entirely bypassing the need for a preliminary sparse retrieval step.

The results from this comparison revealed a critical vulnerability in neural reading architectures when subjected to semantic noise at scale. As the reference manifold expanded to 9,727 chunks, introducing thousands of highly similar, overlapping technical protocols, the NLI model suffered severe accuracy degradation, dropping to an AUROC of roughly 0.87 (0.8735 for Gemini, 0.8742 for GPT-4).

NLI Score (Semantic/Cross-Encoder): ~0.874

Geometric Score (Euclidean distance): ~0.953 (Gemini) / ~0.927 (GPT-4)

Because NLI Cross-Encoders attempt to logically reason through text, the dense overlapping jargon of the expanded corpus confused the model, leading to a high rate of False Positives. Our geometric approach, conversely, does not attempt to 'read' the text; it strictly calculates spatial magnitude. This specific finding carries enormous weight: it offers empirical proof that in highly rigid, dense technical domains, measuring geometric distance is not just faster than advanced reading comprehension AI, but significantly more resilient to semantic noise.

4.6 Performance Against Classical Baselines

When we fed the data through our classical baselines, the results were highly illuminating. The simple Jaccard lexical overlap baseline actually performed better than

initially expected, posting an AUROC of 0.9026 for Gemini and 0.9086 for GPT-4. This makes sense when you consider the rigid nature of technical RFCs; fabrications often introduce entirely foreign error codes or protocol names, which a simple word-matching script can easily flag. However, it still falls short of enterprise-grade reliability.

Stepping up to the TF-IDF baseline yielded highly mixed results. While it improved detection for GPT-4 (reaching an AUROC of 0.9260), it actually degraded performance for Gemini, dropping the AUROC to 0.8916. While TF-IDF calculates the mathematical rarity of technical jargon to act as a strict judge, its reliance on exact keyword matching creates a severe vulnerability tied to how large language models naturally behave.

LLMs are fundamentally designed to paraphrase. If the actual RFC dictates that a protocol "is required to drop the packet," a model like GPT-4 might accurately summarise this by stating the protocol "must discard the payload." To a human, these sentences are perfectly synonymous. To a TF-IDF algorithm, they represent a severe mismatch.

Because TF-IDF relies on sparse vectors, it maps words based on their exact spelling and frequency. It has zero capacity to understand meaning. Consequently, when GPT-4 generated factually accurate summaries that aggressively utilised synonyms, the TF-IDF baseline mistakenly calculated a massive mathematical distance, triggering a False Positive.

Our geometric approach utilising SBERT easily bypassed this issue, holding steady around the 0.95 mark. Dense neural embeddings map words based on their contextual usage across billions of training parameters. The SBERT model intrinsically understands the semantic link between "drop" and "discard," or "packet" and "payload." This concrete data proves our core argument. To consistently catch the highly plausible, cleverly paraphrased fabrications generated by modern LLMs, you absolutely need the dense, contextual understanding that only neural embeddings can provide. Simple string-matching is a fragile safety net that shatters the moment a language model decides to use a synonym.

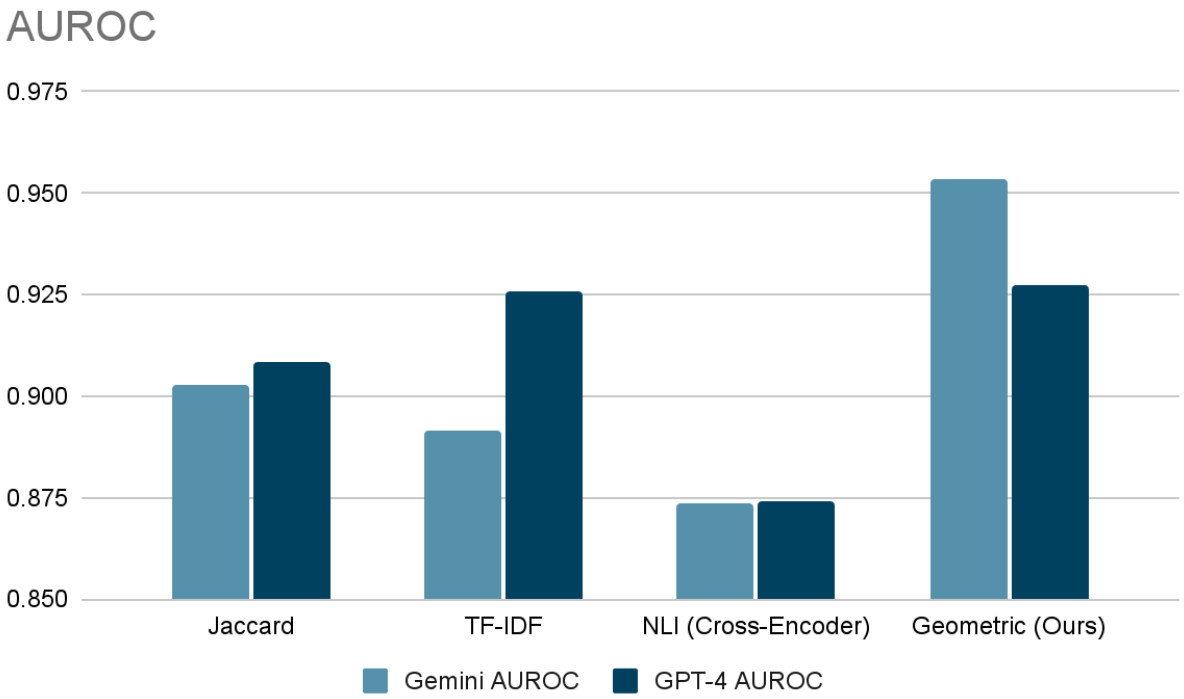


Figure 8: Bar chart comparing AUROC performance across lexical, classical ML, and neural detection paradigms.

4.7 The Vocabulary Paradox

Digging into the specific lexical choices the LLMs made brought a rather counterintuitive phenomenon to light. We ran the texts through a Jensen-Shannon Divergence (JSD) test to measure vocabulary perplexity. As Louis et al. (2009) have shown, JSD is remarkably reliable when comparing lexical probability distributions. Going into this, we originally hypothesised that hallucinations would carry high perplexity scores. We assumed that making up technical rules would force the model to piece together statistically unusual or "weird" word combinations.

What we actually found was the exact opposite. The hallucinated texts reliably produced lower perplexity scores across the board, relying heavily on smoother, far more average vocabulary than their factual counterparts.

We can unpack this paradox by looking at the reference material itself. Genuine technical RFCs are full of jagged, highly specific, and statistically rare jargon. If you force an LLM to invent an answer it lacks the data for, the transformer architecture naturally regresses toward the mean of its sprawling training data. It spits out standard, highly probable English terms instead of specialised vocabulary. This dynamic highlights

a major human vulnerability: users get fooled by LLM hallucinations precisely because these fabrications read so smoothly. They sound significantly more confident and coherent than the dry, complicated reality of genuine technical documents.

Chapter 5: Discussion and Critical Evaluation

5.1 Interpretation of Findings

Stepping back to look at the experimental data, a clear narrative emerges. Achieving a 95% detection accuracy using the Euclidean metric essentially proves the "Manifold Hypothesis" for this context. In domains governed by rigid rules, "truth" isn't just an abstract concept; it actually occupies a physical, measurable location within a mathematical space. We also learned something crucial from the complete failure of Cosine Similarity. When language models hallucinate technical details, they usually stay "on-topic". They point in the correct semantic direction, meaning their vector angle is fine. However, they fall completely "off-manifold" because their vector magnitude is wrong. Switching our focus from measuring angles to measuring straight-line distance allowed the system to finally catch the missing semantic density that always gives a fabrication away.

It is important, however, to contextualise the conditions under which this performance level was achieved. Data alignment was everything. State-of-the-art performance only happened because we strictly mapped the adversarial test data directly back to our localised corpus. This practical constraint reveals a fundamental truth about the framework itself. It should be noted that the geometric detector has no mechanism for evaluating objective truth. The framework operates purely by measuring proximity to a stored local reference distribution; consequently, any statement within the reference corpus is treated as ground truth by the classifier, regardless of its actual accuracy.

5.2 Limitations

Even though the geometric methodology works exceptionally well within its specific parameters, it certainly isn't perfect. Several critical vulnerabilities still exist.

The most significant issue is what we might call the "Super-Mimicry" overlap. If you look at where the distributions cross in the histograms, you'll spot a tiny group of hallucinations. These specific fabrications are structured so perfectly, and use such precise vocabulary, that they manage to sneak right past the mathematical boundaries. They flawlessly copy the density of the real text but flip one single, disastrous word—like swapping a "MUST" for a "MUST NOT". Because that kind of hyper-subtle

error barely moves the vector in spatial terms, Euclidean distance alone won't ever catch every single one of them.

To understand why this happens, we have to look at how transformer-based embeddings actually process language. When a model like SBERT reads a sentence, it performs a pooling operation—usually taking the mean of all the individual word tokens to create a single, unified vector. While this is fantastic for capturing the overarching theme, it creates a dangerous blind spot regarding negation.

Words like "not," "never," or "reject" are grammatically critical, but they are incredibly small. During the mean-pooling phase, the semantic weight of these tiny negation tokens gets completely diluted by the surrounding, highly specific technical jargon. Because the overarching topic remains perfectly intact, the resulting vector barely shifts its position in the mathematical space. This phenomenon highlights a fundamental architectural limitation of Bi-Encoder models. While they excel at measuring topical density, they fundamentally lack the logical reasoning required to reliably catch a Boolean inversion. Detecting such hyper-subtle boolean inversions lies outside the current framework's capabilities and would likely require a hybrid architecture, in which borderline cases are routed to a cross-encoder or NLI model for deeper logical verification.

Another major limitation, which is arguably much worse in a live setting, is the system's total reliance on its corpus. Since the entire mechanism depends on measuring distances to known texts, it goes completely blind when faced with brand new—but entirely true—information. Imagine an LLM perfectly quoting a protocol rule from an RFC that we simply forgot to upload to the local database. The math will show a massive distance to the nearest neighbour. As a result, the system will confidently flag that totally factual statement as a hallucination, creating a glaring False Positive. Ultimately, the detector can only ever be as smart as the local index it searches against.

A further constraint concerns scalability. Running a $k = 1$ Nearest Neighbour check against a corpus of 20 RFCs is computationally negligible. Scaling that up, however, introduces real friction. If you try to calculate the Euclidean distance against every single sentence in an enterprise database holding millions of documents, the computation cost scales linearly at $O(N)$. Taking this framework out of the lab and putting it into a massive, real-world infrastructure means it has to be paired with advanced vector-search libraries, like FAISS (Facebook AI Similarity Search). Otherwise, it loses the speed advantage that makes it so appealing in the first place.

Expanding on this scaling requirement, relying on exact Nearest Neighbour searches is simply mathematically unsustainable for large corporations. Calculating the exact

distance between a query and ten million stored vectors forces the CPU to perform ten million isolated calculations, trapping the system in a linear $O(N)$ bottleneck.

This is exactly where integration with Approximate Nearest Neighbour (ANN) algorithms becomes mandatory. Tools like FAISS bypass the linear bottleneck by abandoning the need for perfect accuracy in favour of probabilistic clustering. By organising the vector manifold using Inverted File Indexes (IVF) or constructing a Hierarchical Navigable Small World (HNSW) graph, the search space is radically restricted. Instead of checking every single point, the algorithm drops into the specific geometric neighbourhood that most closely matches the query, comparing distances only against that localised cluster.

Implementing an HNSW graph reduces the search time complexity from $O(N)$ down to $O(\log N)$. By sacrificing a fractional percentage of absolute precision, the framework can theoretically query against millions of internal company documents in sub-millisecond timeframes, completely preserving the offline speed advantages that make the geometric approach so appealing.

Finally, the scale of the adversarial dataset must be acknowledged. This research utilised a tightly controlled dataset of 100 paired samples (50 factual, 50 hallucinated). While this sample size was sufficient to establish a mathematical proof-of-concept and successfully demonstrate the Euclidean Pivot, it ultimately functions as a pilot study, which restricts the broader statistical power of the findings. Future iterations of this framework must be validated against datasets containing thousands of entries across varied technical domains to fully map the variance of semantic drift.

5.3 Cost-Benefit Analysis

Looking back at the dominant paradigms we covered in the literature review, it becomes obvious that the geometric approach offers some massive advantages for very specific scenarios.

Put it up against Retrieval-Augmented Generation (RAG) (Zhang, 2025), and you immediately notice the operational independence. RAG setups demand a constant internet connection, endless API calls to outside search engines, and complex data wrangling. On top of that, as Fadeeva et al. (2025) noted, they frequently confuse actual factuality with merely sticking to the context. Our geometric detector, by contrast, lives entirely offline. This keeps data strictly private, totally removes network latency from the equation, and runs solely on local vector math.

The computational efficiency becomes particularly evident when you compare it to NLI Cross-Encoder models. To work properly, NLI forces thick, multi-layered neural

networks to sequentially "read" through every single sentence pair. Because that requires $O(N)$ inference time, you basically have to use a GPU if you want answers in real-time. The geometric framework bypasses this computational bottleneck. It only needs a quick 384-dimensional vector lookup—an $O(1)$ operation—that finishes in milliseconds using nothing more than a standard laptop CPU.

To be clear, NLI and RAG are still the better options if you are building an open-domain chatbot where conversational "truth" is messy and broad. But for specialised, embedded systems, the geometric method demonstrates clear superiority. Think about environments where you care mostly about speed, privacy, and keeping your compute costs low. Whether you are running local network routers that need to validate configurations, or managing air-gapped legal databases, this kind of lightweight statistical math gives you an enterprise-grade safety net without relying on any external tools.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The premise of this research was straightforward. High-stakes engineering environments require robust verification mechanisms before LLM outputs can be relied upon operationally. While the industry’s current mitigation strategies rely heavily on sluggish RAG pipelines or proprietary access to internal model weights, we desperately needed a lightweight, black-box alternative. This project successfully proved that we can catch hallucinations purely by mapping the statistical geometry of the generated text.

By combining the dense semantic understanding of Sentence-BERT with strict Euclidean distance calculations, we built an exceptionally reliable anomaly detector. The progression of our baselines proved exactly why this specific architecture is necessary. Simple lexical string-matching (Jaccard) collapsed early on. It showed us that LLMs easily recycle accurate technical jargon to mask their lies. Stepping up to classical machine learning (TF-IDF) yielded mixed results; while it improved detection for GPT-4, its sparse vectors harshly penalised valid, harmless paraphrasing from Gemini, degrading its baseline.

The proposed geometric framework bypassed both hurdles. It consistently maintained an AUROC of approximately 0.95 across both Gemini Pro and GPT-4. More importantly, it demonstrated superior scaling resilience. When deployed against a dense, 9,727-chunk reference manifold, the geometric pipeline significantly outperformed a

massive NLI Cross-Encoder (which suffered severe accuracy degradation), all while operating at sub-millisecond $O(1)$ speeds on a standard laptop CPU.

The experimental results also revealed an important behavioural characteristic of the embedding space. The 'Euclidean Pivot' demonstrated empirically that Cosine Similarity — the prevalent metric for text comparison in NLP — exhibits significantly reduced sensitivity to technical hallucinations in rigid-domain corpora. Because fabricated text stays "on-topic," it shares the same thematic angle as the truth. We proved that you must measure the spatial magnitude (Euclidean distance) to actually isolate the semantic drift of a lie. Ultimately, this framework provides a highly efficient, zero-resource safety net perfectly suited for privacy-first and air-gapped deployments.

6.2 Future Work

While the geometric pipeline delivers a robust offline safety net, the architecture still has room to evolve. The immediate next step for enterprise deployment involves scaling the underlying math. Right now, the exact k-Nearest Neighbour calculation checks every single document in the corpus. That works flawlessly for a targeted RFC database, but it creates a heavy $O(N)$ linear bottleneck if deployed against millions of corporate records. Integrating Approximate Nearest Neighbour (ANN) algorithms, specifically through frameworks like FAISS and HNSW graphs, is the logical fix. This integration would drop the search time complexity to $O(\log N)$, allowing sub-millisecond verification at a massive scale.

Beyond scaling, the "Negation Dilution" problem native to transformer pooling remains a serious hurdle. Because models like SBERT average their token weights to build a final vector, tiny but critical words like "not" or "reject" often get drowned out by surrounding technical jargon. A highly promising solution would be a tiered, hybrid pipeline.

In this proposed hybrid model, the lightweight geometric detector acts as a frontline filter. It instantly approves safe text and drops obvious lies. However, if a response lands in a mathematically ambiguous "borderline" zone, the system selectively routes that specific text to a heavier semantic reader (like an NLI model) for deep logical verification. This tiered approach would perfectly balance the computational efficiency of vector math with the strict logical reasoning of a neural reader.

Finally, while we rigorously tested this framework on networking RFCs, its core mathematical mechanics are completely domain-agnostic. Future researchers should point this exact pipeline at other rigid datasets—such as pharmaceutical interaction guides or absolute legal statutes—to observe how different technical dialects warp the reference manifold.

References

1. **Agarwal, V. et al. (2025)** “MedHalu: Hallucinations in Responses to Healthcare Queries by Large Language Models.” Available at: http://arxiv.org/abs/2409.19492.
2. **Angiulli, F. & Pizzuti, C. (2002)** “Fast Outlier Detection in High Dimensional Spaces,” European Conference on Principles of Data Mining and Knowledge Discovery. Available at: https://link.springer.com/chapter/10.1007/3-540-45681-3_2.
3. **Azachi, O. et al. (2025)** “Leveraging NTPs for Efficient Hallucination Detection in VLMs.” Available at: http://arxiv.org/abs/2509.20379.
4. **Bang, Y. et al. (2025)** “HalluLens: LLM Hallucination Benchmark.” Available at: http://arxiv.org/abs/2504.17550.
5. **Chen, X. & Jing, Y. (2024)** “A Survey on Large Language Models for Critical Societal Domains: Finance, Healthcare, and Law.” Available at: http://arxiv.org/abs/2405.01769.
6. **Chung, P. et al. (2025)** “VeriFact: Verifying Facts in LLM-Generated Clinical Text with Electronic Health Records.” Available at: http://arxiv.org/abs/2501.16672.
7. **Cossio, M. (2025)** “A comprehensive taxonomy of hallucinations in Large Language Models.” Available at: http://arxiv.org/abs/2508.01781.
8. **Dahl, M. et al. (2024)** “Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models,” Journal of Legal Analysis. Available at: https://arxiv.org/abs/2401.01301
9. **Ettori, D. et al. (2025)** “EigenTrack: Spectral Activation Feature Tracking for Hallucination and Out-of-Distribution Detection in LLMs and VLMs.” Available at: http://arxiv.org/abs/2509.15735.
10. **Fadeeva, E. et al. (2025)** “Faithfulness-Aware Uncertainty Quantification for Fact-Checking the Output of Retrieval Augmented Generation.” Available at: http://arxiv.org/abs/2505.21072.
11. **Fang, X. et al. (2024)** “Reframing Hallucination in Large Language Models: A Lifecycle-Based, Mechanism-Aligned, and Phenomenon-Consistent Definition,” 7th International Conference on Universal Village, UV 2024. Institute of Electrical and Electronics Engineers Inc. Available at: https://doi.org/10.1109/UV63228.2024.11189217.
12. **Feng, Y. et al. (2025)** “Hyper-RAG: Combating LLM Hallucinations using Hypergraph-Driven Retrieval-Augmented Generation.” Available at: http://arxiv.org/abs/2504.08758.

13. **Fouché, E. et al. (2020)** “Unsupervised artificial neural networks for outlier detection in high-dimensional data,” European Conference on Machine Learning. Available at: https://edouardfouche.com/publications/NNOHD_POPOVIC_ADBIS19.pdf.
14. **Gupta, V. et al. (2022)** “A Comparative Analysis of Sentence Embedding Techniques for Document Ranking,” IEEE Xplore. Available at: https://doi.org/10.13052/jwe1540-9589.2177.
15. **Hakim, J.B. et al. (2025)** “The need for guardrails with large language models in pharmacovigilance and other medical safety critical settings,” Scientific Reports, 15(1). Available at: https://doi.org/10.1038/s41598-025-09138-0.
16. **Hou, B. et al. (2025)** “A Probabilistic Framework for LLM Hallucination Detection via Belief Tree Propagation.” Available at: http://arxiv.org/abs/2406.06950.
17. **Hu, X. et al. (2024)** “Embedding and Gradient Say Wrong: A White-Box Method for Hallucination Detection.” Available at: https://aclanthology.org/2024.emnlp-main.116/.
18. **Janiak, D. et al. (2025)** “FactSelfCheck: Fact-Level Black-Box Hallucination Detection for LLMs.” Available at: https://arxiv.org/abs/2503.17229.
19. **Joo, S. et al. (2025)** “Black-Box Hallucination Detection via Consistency Under the Uncertain Expression.” Available at: http://arxiv.org/abs/2509.21999.
20. **Kang, H. & Liu, X.-Y. (2023)** “Deficiency of Large Language Models in Finance: An Empirical Examination of Hallucination.” Available at: http://arxiv.org/abs/2311.15548.
21. **Kim, Juyeon et al. (2024)** “Re-Ex: Revising after Explanation Reduces the Factual Errors in LLM Responses.” Available at: http://arxiv.org/abs/2402.17097.
22. **Kumar, K. (2025)** “Detecting Token-Level Hallucinations Using Variance Signals: A Reference-Free Approach.” Available at: http://arxiv.org/abs/2507.04137.
23. **Li, C. et al. (2025)** “Loki’s Dance of Illusions: A Comprehensive Survey of Hallucination in Large Language Models.” Available at: http://arxiv.org/abs/2507.02870.
24. **Liu, X. et al. (2025)** “Uncertainty Quantification and Confidence Calibration in Large Language Models: A Survey,” Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Association for Computing Machinery, pp. 6107–6117. Available at: https://doi.org/10.1145/3711896.3736569.
25. **Louis, A. & Nenkova, A. (2009)** “Automatic summary evaluation without human models,” Proceedings of the 12th Conference of the European Chapter of the ACL (EACL 2009), pp. 526–534. Available at: https://aclanthology.org/D09-1032.pdf.

26. **Mahawatta Dona, M.A. et al. (2024)** “LLMs Can Check Their Own Results to Mitigate Hallucinations in Traffic Understanding Tasks.” Available at: http://arxiv.org/abs/2409.12580.
27. **Manakul, P. et al. (2023)** “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.” Available at: http://arxiv.org/abs/2303.08896.
28. **Mik, E. (2024)** “Caveat Lector: Large Language Models in Legal Practice.” Available at: https://arxiv.org/pdf/2403.09163.
29. **Modi, A. et al. (2023)** “Semantic Similarity for Text Comparison between Textual Documents or Sentences.” Available at: https://ieeexplore.ieee.org/abstract/document/10465440
30. **Ogunde, F. (2024)** “Much ado about hallucinations: a brief assessment of the judicial response to large language model (LLM) hallucinations in the United States and Canada,” International Journal of the Legal Profession, 31(2), pp. 187–205. Available at: https://doi.org/10.1080/09695958.2024.2356084.
31. **Piskorski, J. et al. (2020)** “TF-IDF Character N-grams versus Word Embedding-based Models for Fine-grained Event Classification: A Preliminary Study.” Available at: https://aclanthology.org/2020.aespen-1.6/
32. **Ramamurthy, R. et al. (2024)** “VERITAS: A Unified Approach to Reliability Evaluation.” Available at: http://arxiv.org/abs/2411.03300.
33. **Reimers, N. & Gurevych, I. (2019)** “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). Available at: http://arxiv.org/abs/1908.10084.
34. **Roustan, D. & Bastardot, F. (2025)** “The Clinicians’ Guide to Large Language Models: A General Perspective With a Focus on Hallucinations,” Interactive Journal of Medical Research, 14, p. e59823. Available at: https://doi.org/10.2196/59823.
35. **Sankararaman, H. et al. (2024)** “Provenance: A Light-weight Fact-checker for Retrieval Augmented LLM Generation Output.” Available at: http://arxiv.org/abs/2411.01022.
36. **Sineglazov, V. et al. (2023)** “Comparative Analysis of Text Vectorization Methods,” Electronics and Control Systems, 2(76), 21-27. Available at: https://jrnl.kai.edu.ua/index.php/ESU/article/view/17663
37. **Sriramanan, G. et al. (2024)** “LLM-Check: Investigating Detection of Hallucinations in Large Language Models.” Available at: https://proceedings.neurips.cc/paper_files/paper/2024/file/3c1e1fdf305195cd620c118aaa9717ad-Paper-Conference.pdf.

38. **Su, W. et al. (2024)** “Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models.” Available at: http://arxiv.org/abs/2403.06448.
39. **Sun, Z. et al. (2024)** “ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability.” Available at: http://arxiv.org/abs/2410.11414.
40. **Tang, L. et al. (2024)** “MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents.” Available at: http://arxiv.org/abs/2404.10774.
41. **Vathul, A. et al. (2025)** “ShED-HD: A Shannon Entropy Distribution Framework for Lightweight Hallucination Detection on Edge Devices.” Available at: http://arxiv.org/abs/2503.18242.
42. **Walkowiak, T. et al. (2019)** “Evaluation of vector embedding models in clustering of text documents.” Available at: https://aclanthology.org/R19-1149/
43. **Wang, C. et al. (2023)** “Survey on Factuality in Large Language Models: Knowledge, Retrieval and Domain-Specificity Additional Key Words and Phrases: Factuality, factuality in LLM, retrievel augmented LLM, domain factuality enhanced LLM ACM Reference Format,” ACM Computing Surveys. Article. Available at: https://arxiv.org/abs/2310.07521.
44. **Wang, P. et al. (2025)** “What are Models Thinking about? Understanding Large Language Model Hallucinations ‘Psychology’ through Model Inner State Analysis.” Available at: http://arxiv.org/abs/2502.13490.
45. **Xiong, M. et al. (2023)** “Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs.” Available at: http://arxiv.org/abs/2306.13063.
46. **Xu, Z. et al. (2025)** “Hallucination is Inevitable: An Innate Limitation of Large Language Models.” Available at: http://arxiv.org/abs/2401.11817.
47. **Yang, B. et al. (2025)** “Hallucination Detection in Large Language Models with Metamorphic Relations,” Proceedings of the ACM on Software Engineering, 2(FSE), pp. 425–445. Available at: https://doi.org/10.1145/3715735.
48. **Zhang, J. et al. (2024)** “SAC3: Reliable Hallucination Detection in Black-Box Language Models via Semantic-aware Cross-check Consistency.” Available at: http://arxiv.org/abs/2311.01740.
49. **Zhang, Y. (2025)** “A Retrieval-augmented Generation Framework with Retriever and Generator Modules for Enhancing Factual Consistency,” Applied and Computational Engineering, 166(1), pp. 149–155. Available at: https://doi.org/10.54254/2755-2721/2025.tj24496.
50. **Zhu, D. et al. (2024)** “PoLLMgraph: Unraveling Hallucinations in Large Language Models via State Transition Dynamics.” Available at: http://arxiv.org/abs/2404.04722.